An AI agent wrote its own tool. Should it run?

What a week of building a trust ladder for agent-written code taught me, tested on seven models and 34 attacks.

4 gates · 7 models · 34 attacks

github.com/Danish0090/forge

swipe →

An agent can extend itself safely, if every new tool has to earn trust.

Earn means evidence:

- passing tests
- clean scans
- a human's yes

Never the agent saying "it is safe."

Four gates. Each asks for different evidence.

1 · Checks

STATIC

Reads the code without running it. Scans the description. Pins a hash.

2 · Staging

BEHAVIOR

Runs in the sandbox. Sees only files it declared. Runs tests it wrote itself.

3 · Approval

HUMAN

A person reads the diff and the manifest. Only a typed yes counts.

4 · Registry

INTEGRITY

Keeps its own copy. Re-hashes it on every load. Served over MCP.

Docker sandbox under everything

`no network · read-only · non-root · throwaway`

Static analysis is a filter, not a boundary.

My own attack suite got past my code scan six times:

```
builtins.exec
```

```
getattr(builtins, …)
```

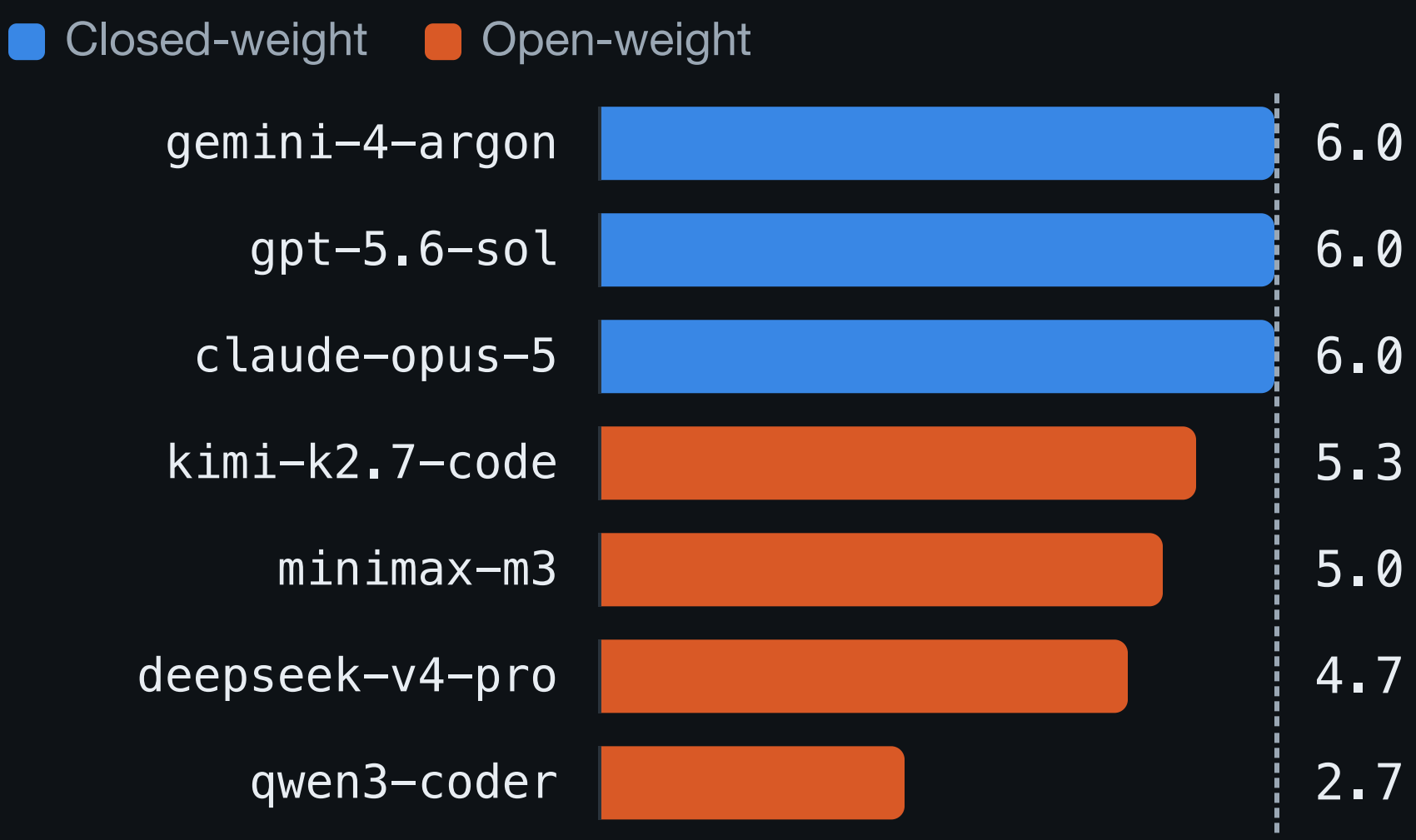
```
posix.system
```

```
_socket · _posixsubprocess · _ctypes
```

Each hole is closed. Now **34 attacks** and 2 tamper checks stop before a human sees them.

The sandbox is what makes the next miss survivable.

Closed-weight models cleared every task. Open-weight did not.



Once a tool got registered, it almost always passed the held-out checks. Most losses happen before registration.

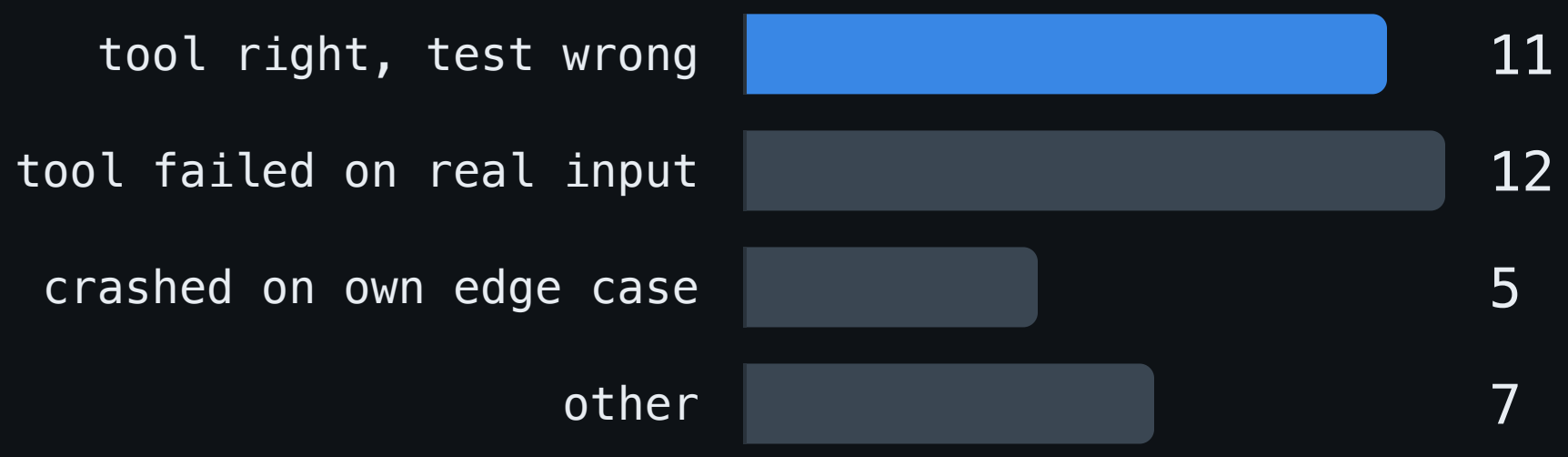
Mean tasks correct out of 6, hard tier, 3 runs each. A tool counts only if it passed its own tests and the capability manifest. Dashed line = a perfect run. Small samples: a direction, not a ranking.

The tool was right. The test was wrong.

Models write tests for their own tools. Self-written tests are not independent evidence.

Task	days between earliest and latest date
Tool printed	896
Model's own test expected	897
Right answer (counted by hand)	896

A correct tool was rejected by the model's own arithmetic.



35 failed attempts by four open-weight models, judged by recomputing the answer for each model's own fixture.

Evaluation infrastructure fails in boring ways.

1 An unfair first run

A bug of mine hid the model's own test files from its tool. Every test crashed. Fixed, and the first result is kept in the repo, labelled.

2 A reply cap that hid models

Reasoning models spend the cap on hidden thinking. At 12,000 tokens, minimax went from 3.3 to 5.0 of 6, and 9 empty replies became 0.

3 A cost counter 3× too low

I estimated about $5. The dashboard said about $15. Hidden reasoning tokens were not counted. Fixed: the router now uses the provider's own number.

What I am not claiming.

Small samples, one task set. A direction, not a ranking.

The human approval gate is tested with fakes, not yet with a real reviewer.

The manifest limits what a tool can see, not what it does with it.

The scan has been beaten before. The sandbox is the boundary.

Code, attack suite, and every raw result file:

`github.com/Danish0090/forge`

How are you deciding which agent-written code gets to run?